UNIVERSIDAD TECNOLÓGICA NACIONAL - FACULTAD REGIONAL MENDOZA

ARQUITECTURA DE LAS COMPUTADORAS II

CPU SPARC

MATÍAS BARDOSSY

DAMIÁN BLANGETTI

GABRIEL CAFFARATTI

AÑO 2004

RESUMEN

En primer lugar veremos como funciona el CPU SPARC, desde la longitud de sus buses hasta la codificación de sus instrucciones, haciendo referencia a sus registros y a la manera en que el Procesador accede a los mismos.

Luego estudiaremos cada una de sus instrucciones en detalle, y sus modos de direccionamiento, explicaremos como se distribuyen los bits en las mismas para los distintos modos de direccionar las instrucciones.

También veremos y daremos una breve explicación de cada una de las partes que compone al CPU SPARC y hablaremos del Sistema Operativo Solaris que aprovecha al máximo su utilización.

Por último daremos algunos datos técnicos del Procesador UltraSPARC III y UltraSPARC IV y haremos una comparación con un Pentium 4. Finalizando con algunas recomendaciones y conclusiones.

OBJETIVOS

- Conocer la estructura y las partes de un CPU SPARC.
- Comprender el funcionamiento del CPU SPARC.
- Observar las diferencias que presenta el CPU SPARC, respecto con otros CPUs.
- Conocer sus instrucciones y la estructura de las mismas.
- Observar las aplicaciones y usos del CPU SPARC.

INTRODUCCIÓN

En 1987, Sun Microsystems anunció una arquitectura RISC abierta denominada SPARC (Scalable Processor ARChitecture, o en español Arquitectura de Procesador Escalable), la cual sería la base de futuros productos de la empresa.

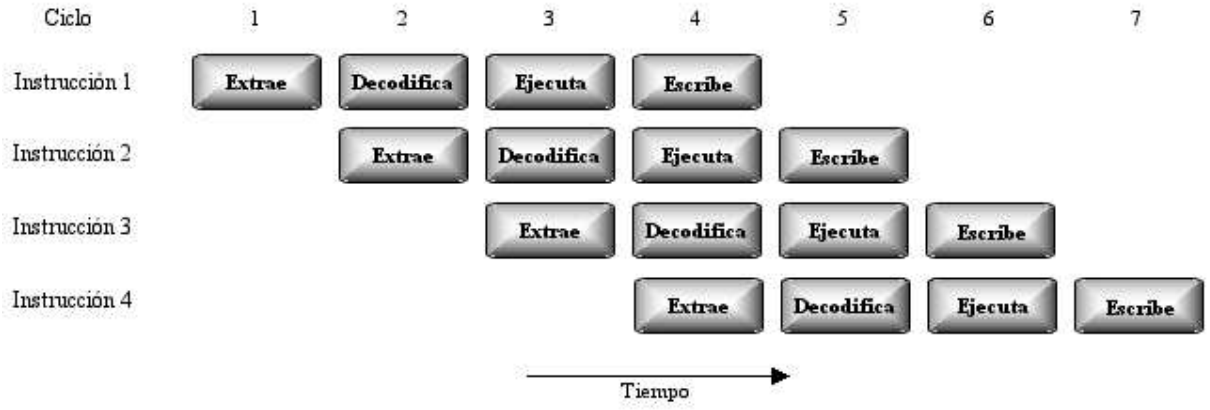
Alrededor de media docena de distribuidores de SPARC obtuvieron la licencia para fabricar pastillas SPARC usando diferentes tecnologías (CMOS, ECL, GaAs, Arreglos de compuertas, VLSI, etc.). La intención fue alentar la competencia entre los distribuidores de pastillas, a fin de mejorar en el desempeño, reducir precios y hacer el intento de establecer la arquitectura SPARC como estándar en la industria.

La tecnología Sun, con respecto al SPARC, comenzó con una arquitectura de 32 bits, la cual es la que usan la mayoría de los procesadores fabricados actualmente, pero luego se expandió a una tecnología de 64 bits, lo cual significa el doble de tamaño de los registros y de bus de datos. Nosotros nos centraremos en la arquitectura inicial de Sun de 32 bits.

DESCRIPCIÓN DEL HARDWARE

Descripción Funcional del Conjunto

La arquitectura SPARC se ha definido con mucho cuidado para permitir la implantación de procesamiento en serie muy avanzado. Entre otros aspectos, define retardos en carga y almacenamiento, bifurcaciones, llamadas y retornos. La implantación típica tiene un procesamiento en serie de cuatro etapas (como se muestra en la siguiente figura). Durante el primer ciclo se extrae de la memoria la palabra de la instrucción; en el segundo se decodifica; durante el tercero se ejecuta; por último en el cuarto ciclo se escribe el resultado otra vez en la memoria.



El espacio de direcciones de los procesadores SPARC es lineal y no segmentado. Por otra parte el bus de direcciones es de 32 bits lo que permite direccionar hasta 2^{32} bytes = 4 *Gbytes* de memoria virtual. Es decir que para especificar una dirección de memoria son necesarios 32 bits.

Otra característica importante de los procesadores SPARC es que son procesadores RISC (Computadora con reducido conjunto de instrucciones). Para que un procesador sea considerado RISC debe cumplir, entre otras cosas, que **el tamaño de sus instrucciones no sea variable**, y que en consecuencia estas se completen en un solo ciclo (entendiendo por ciclo la extracción de los operandos de un registro, colocarlos en el bus, ejecutarlos en la ALU, y guardar el resultado en un registro)**.**

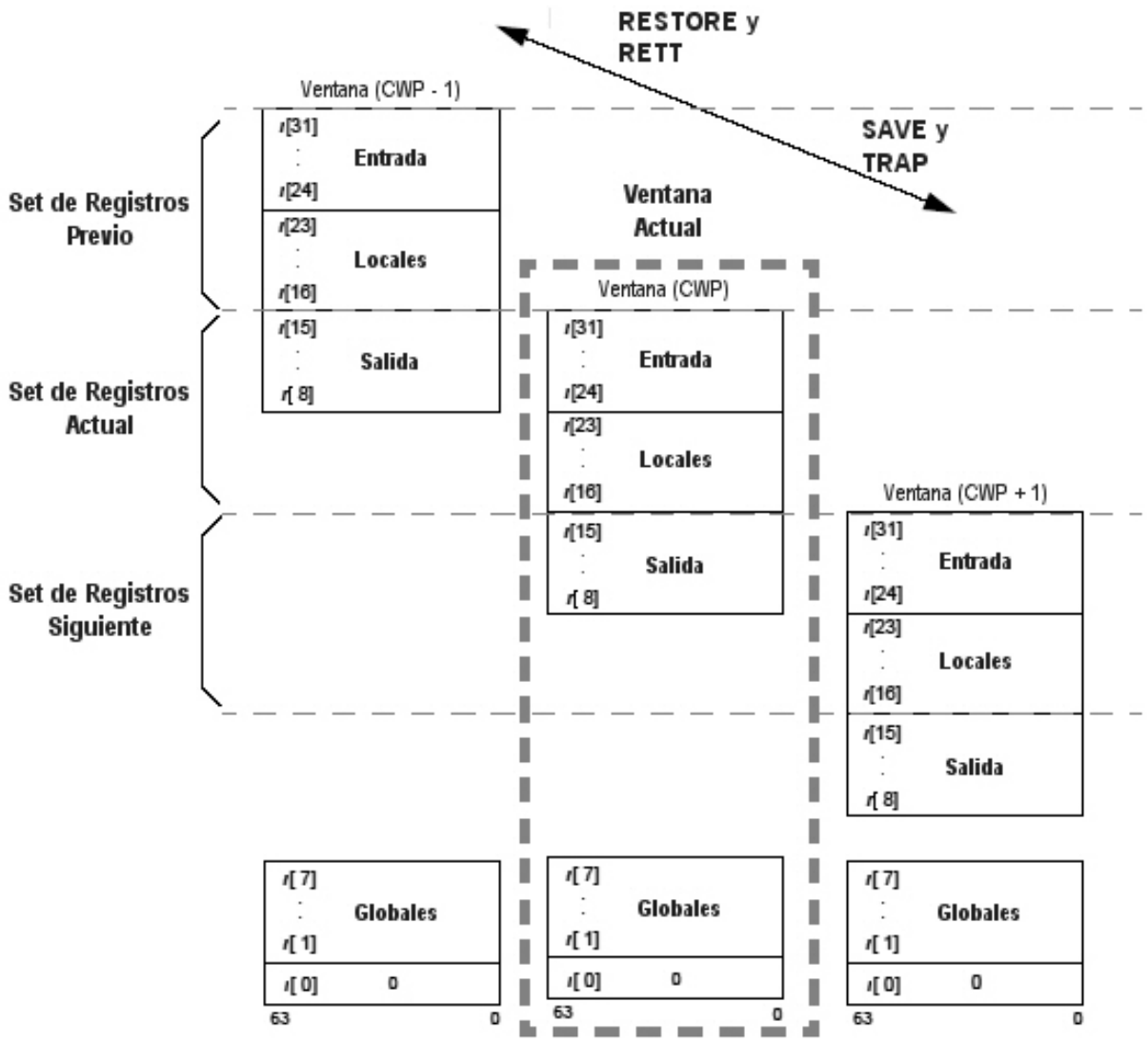
De lo anterior queda claro que no es posible, en los procesadores SPARC que estudiamos, codificar una dirección de memoria absoluta de 32 bits junto con la instrucción que la utilizará ya que no queda espacio para colocar el valor de la dirección más los bits necesarios para codificar la instrucción.

Para realizar esto el procesador SPARC provee una instrucción especial llamada **SETHI** que permite colocar un valor de hasta 22 bits en los 22 bits más significativos de algún registro. De esta forma luego utilizando adecuadamente la instrucción **OR** es posible colocar los 10 bits menos significativos de la constante en los 10 bits menos significativos del registro. De este modo existen instrucciones para cargar y almacenar cantidades de 8, 16 ,32 y 64 bits, en los registros de 32 bits, usando en este último caso dos registros consecutivos.

Una diferencia de los procesadores CISC (Computadora con complejo conjunto de instrucciones) a los procesadores RISC es que una gran parte no poseen “stack”. Dado que una de las características que se desea en un procesador es rapidez, debe tenerse en cuenta que las instrucciones que extraen sus operandos de los registros y almacenan los resultados también en ellos, se pueden manejar en un solo ciclo. Sin embargo, aquellas que cargan información desde la memoria o almacenan en ésta consumen demasiado tiempo. Es por ello que los procesadores RISC, incluyendo el SPARC, poseen una alta cantidad de registros internos de tal manera que las instrucciones ordinarias tienen operandos provenientes de los mismos. No obstante algunas instrucciones deben hacer referencia o uso de la memoria, y debido a esto es que yacen incorporadas en la arquitectura las instrucciones especiales LOAD/STORE.

Los procesadores SPARC implementan un “sistema de ventanas”. Cada ventana son 24 registros (a los cuales se le suman 8 más que son globales, o sea, accesibles por todas las ventanas; la suma de 32 registros es visible en todo momento) que pertenecen a un procedimiento o subprograma. A medida que se van creando nuevas ventanas los registros de salida de la ventana actual se solapan sobre los de entrada de la ventana siguiente. Los

registros locales y de salida de la ventana actual son diferentes a los registros locales y de salida de la ventana siguiente. Esta figura lo muestra esquemáticamente.



En la figura, CWP es el índice de la ventana actual. El comando SAVE y las TRAPS decrementan dicho índice, y RESTORE y RETT lo incrementan.

Para verificar si un número de ventana es válido se utiliza el registro WIN (Window Invalid Mask) de 32 bits. Si el bit i-ésimo está en 1 indica que esa ventana inválida y un intento de acceder a la misma genera un TRAP "window_overflow".

El registro G0 está fijo por medio del hardware con el valor 0. En él, los almacenamientos no alteran su valor. Los registros G1 a G7 son globales y pueden contener variables enteras, apuntadores a tablas u otros datos importantes.

Los registros de salida son de O0 a O7 y son usados por los procedimientos para pasar parámetros a los procedimientos que llaman. El primer parámetro va al registro O0, el siguiente al O1 y así sucesivamente. El registro O6 (AE) se utiliza como apuntados a la pila de memoria. La pila se usa para los parámetros que exceden los registros, espacio de asignación dinámica de la pila, ventanas apiladas en memoria, así como para salvar registros de punto flotante, apuntadores a buffers en donde los procedimientos llamados pueden enviar

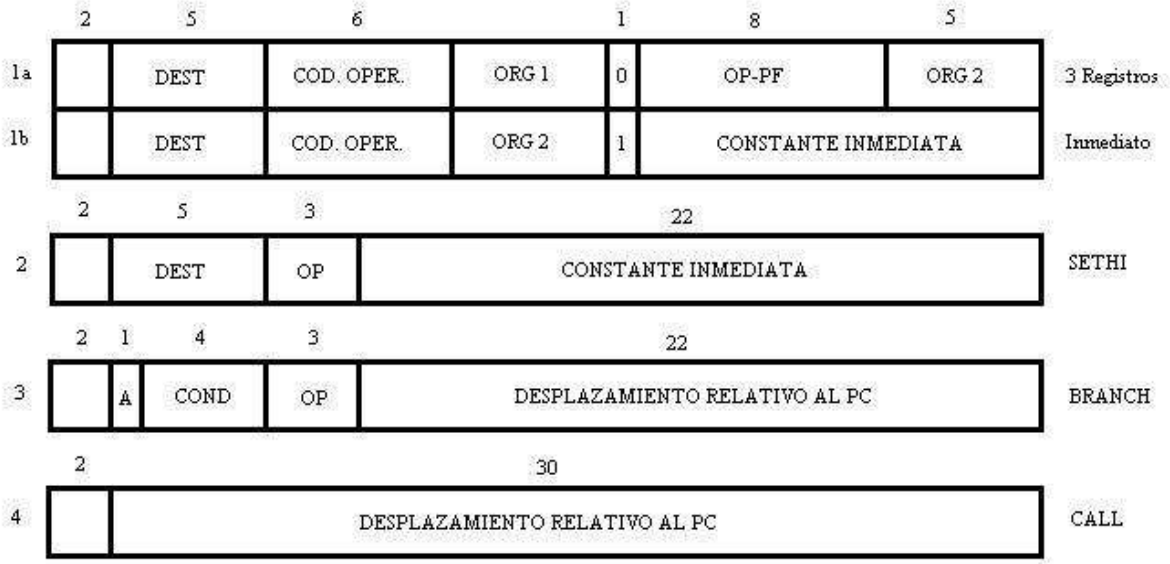
estructuras y arreglos de datos, etc. La instrucción CALL (llamada) deposita la dirección de retorno en el registro O7. Los 8 registros para variables locales L0 a L7, se pueden usar en la forma que el programador o el compilador considere más conveniente. Las 8 variables de entrada I0 a I7 son los parámetros que son pasados al procedimiento actual por el que le llama: Los registros no utilizados pueden contener variables locales adicionales. I6 (FP) es el apuntador de marco que se usa para direccionar variables en el marco de pila. A diferencia de SP que solo puede cambiar mientras el procedimiento se ejecuta, FP apunta la misma palabra en memoria durante toda la ejecución del procedimiento, siendo más apropiado para la indexación que el SP. La dirección de retorno del procedimiento se encuentra contenida en el registro I7.

INSTRUCCIONES

Se puede utilizar LOAD y STORE en cualquiera de los dos formatos 1a y 1b, en los que DEST sería el registro a cargar o almacenar, los 19 bits de orden inferior de la instrucción determinan cómo se realiza el calculo de la dirección efectiva de memoria. Se proporcionan dos modos de direccionamiento (formatos 1a y 1b):

1- Dirección de memoria = ORG1 + ORG2
2- Dirección de memoria = ORG1 + constante de 13 bits con signo codificado en complemento a dos, perteneciente al rango de intervalo cerrado –4096, 4095.

Los 32 registros se pueden usar tanto como registros de origen o como de destino (ORG1 u ORG2).



Formato de instrucción SPARC

Volviendo al problema sobre cómo colocar en cualquiera de ellos un valor de 32 bits si cualquier instrucción utiliza como máximo 32 bits, suponiendo que se desea colocar en el registro %4 el valor FFFF0000h. La instrucción debería utilizar algunos bits para codificar el 'mov' y qué registro quiero utilizar con lo cual quedan menos de 32 bits para almacenar el

valor inmediato FFFF0000h. La respuesta a la pregunta es claramente que no se puede realizar con una sola instrucción.

Dijimos que para realizar esta tarea el procesador SPARC provee una instrucción especial llamada **SETHI** que permite colocar un valor de hasta 22 bits en los 22 bits más significativos de algún registro. Y que utilizando la instrucción **OR** es posible colocar los 10 bits menos significativos de la constante en los 10 bits menos significativos del registro.

Veamos ahora cómo sería el código para realizar "r4=FFFF0000h" :

```
VALOR_32    EQU   FFFF0000h
V_HIGH_22   EQU   VALOR_32 / 1024    ; 22 bits más significativos
                                       de la etiqueta VALOR_32
V_LOW_10    EQU   VALOR_32 % 1024    ; 10 bits menos significativos
                                       de la etiqueta VALOR_32

SETHI V_HIGH_20 , %4            ; En los 22 bits más significativos de %4 se
                                  coloca el valor V_HIGH_20 en los 10 bits
                                  menos significativos coloca ceros
OR %4 , #V_LOW_10 , %4          ; Los 10 bits menos significativos de
                                  VALOR_32 son colocados en los 10 bits
                                  menos significativos de %4
```

La forma de direccionar memoria utilizada por los procesadores SPARC es a través de registros y tiene la siguiente sintaxis:

#desp13(*registro*) o (1)

(*registro)(registro)* *(2)*

Aquí *desp13* es un numero de 13 bits con signo codificado en complemento a 2, es decir que pertenece al rango [-4096,4095] y registro es cualquiera de los 32 registros del procesador.

La semántica de la expresión (1), es decir a qué dirección representa, está dada por:

(*registro*)+*desp13*

y la semántica de la expresión (2) es

(*registro*1) + (*registro*2)

(registro) es el valor almacenado en el registro

Ejemplos:

#57(%4) ; contenido de %4 + 57

#-10(%15) ; contenido de %15 - 10

(%4)(%15) ; contenido de %4 + contenido de %15

A continuación procederemos a mencionar cada una de las instrucciones que posee un procesador SPARC y la función que cumplen:

ARITMÉTICAS

- **ADD**[**cc**]

Formato: ADD op_1, op_2 , op_3

ADDcc op_1, op_2 , op_3

Tipo Args: (reg) (*reg_o_imm*) (reg)

Lógica: $op_3 \leftarrow op_1 + op_2$

Descripción: Suma op_1 a op_2 y almacena el resultado en op_3.

Banderas: Si se utiliza con **cc** afecta todas las flags.

- **ADDX[cc]**

Formato: ADDX op_1, op_2, op_3
ADDXcc op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 + op_2 + c$
Descripción: Suma op_1 a op_2 y el carry y almacena el resultado en op_3.
Banderas: Si se utiliza con **cc** afecta todas las flags.

- **SUB[cc]**

Formato: SUB op_1, op_2 , op_3
SUBcc op_1, op_2 , op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 - op_2$
Descripción: Resta op_2 a op_1 y almacena el resultado en op_3
Banderas: Si se utiliza con **cc** afecta todas las flags.

- **SUBX[cc]**

Formato: SUBX op_1, op_2 , op_3
SUBXcc op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 - op_2 - c$
Descripción: Resta op_2 y el carry a op_1 y almacena el resultado en op_3
Banderas: Si se utiliza con **cc** afecta todas las flags.

LÓGICAS

- **AND[cc]**

Formato: AND op_1, op_2, op_3
ANDcc op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 \wedge op_2$
Descripción: Realiza el "and" bit a bit entre op_1 y op_2 y almacena el resultado en op_3.
Banderas: Si se utiliza con **cc** afecta todas las flags.

- **OR[cc]**

Formato: OR op_1, op_2, op_3
ORcc op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 \vee op_2$
Descripción: Realiza el "or" bit a bit entre op_1 y op_2 y almacena el resultado en op_3.
Banderas: Si se utiliza con **cc** afecta todas las flags.

- **XOR[cc]**

Formato: XOR op_1, op_2 , op_3
XORcc op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 \oplus op_2$
Descripción: Realiza el "xor" bit a bit entre op_1 y op_2 y almacena el resultado en op_3.
Banderas: Si se utiliza con **cc** afecta todas las flags.

DESPLAZAMIENTO

SLL

Formato: SLL op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 \,.\, 2^{op_2}$
Descripción: Realiza un shift a la izquierda de los bits de op_1, op_2 lugares y almacena el resultado en op_3.
Sólo considera los 5 bits más significativos de op_2 para determinar el valor del contador.
Rellena con ceros.
Banderas: No modifica las flags.

SRL

Formato: SRL op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Lógica: $op_3 \leftarrow op_1 / 2^{op_2}$ (División entera)
Descripción: Realiza un shift a la derecha de los bits de op_1, op_2 lugares y almacena el resultado en op_3.
Sólo considera los 5 bits más significativos de op_2 para determinar el valor del contador.
Rellena con ceros.
Banderas: No modifica las flags.

SRA

Formato: SRA op_1, op_2, op_3
Tipo Args: (reg) (*reg_o_imm*) (reg)
Descripción: Realiza un shift a la derecha de los bits de op_1, op_2 lugares y almacena el resultado en op_3.
Sólo considera los 5 bits menos significativos de op_2 para determinar el valor del contador.
Realiza expansión de signo, es decir, rellena con el bit más significativo de op_1.
Banderas: No modifica las flags.

MEMORIA

LDSB

Formato: LDSB op_1, op_2
Tipo Args: (direcc) (reg)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$
Descripción: Carga en op_2 el valor del byte almacenado en la dirección de memoria op_1.
Realiza expansión de signo.
Banderas: No modifica las flags.

LDSH

Formato: LDSH op_1, op_2
Tipo Args: (direcc) (reg)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$

Descripción: Carga en op_2 el valor de la media palabra (2 bytes) almacenada a partir de la dirección de memoria op_1 .
Realiza expansión de signo.
Banderas: No modifica las flags.

- **LDUB**

Formato: LDUB op_1, op_2
Tipo Args: (direcc) (reg)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$
Descripción: Carga en op_2 el valor del byte almacenado en la dirección de memoria op_1 .
No realiza expansión de signo. Rellena con ceros.
Banderas: No modifica las flags.

- **LDUH**

Formato: LDSH op_1, op_2
Tipo Args: (direcc) (reg)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$
Descripción: Carga en op_2 el valor de la media palabra (2 bytes) almacenada a partir de la dirección de memoria op_1 .
No realiza expansión de signo. Rellena con ceros.
Banderas: No modifica las flags.

- **LD**

Formato: LD op_1, op_2
Tipo Args: (direcc) (reg)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$
Descripción: Carga en op_2 el valor de la palabra (4 bytes) almacenada a partir de la dirección de memoria op_1.
Banderas: No modifica las flags.

- **STB**

Formato: STB op_1, op_2
Tipo Args: (reg) (direcc)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$
Descripción: Guarda en el byte ubicado en la dirección de memoria op_2 los 8 bits menos significativos de op_1.
Banderas: No modifica las flags.

- **STH**

Formato: STH op_1, op_2
Tipo Args: (reg) (direcc)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$
Descripción: Guarda en la media palabra (2 bytes) ubicada a partir de la dirección de memoria op_2 los 16 bits menos significativos de op_1.
Banderas: No modifica las flags.

- **ST**

Formato: ST op_1, op_2
Tipo Args: (reg) (direcc)
Lógica: $op_2 \leftarrow MEMORIA\ (op_1)$
Descripción: Guarda en la palabra (4 bytes) ubicada en la dirección de memoria op_2 el valor de $op1$.
Banderas: No modifica las flags.

- **SWAP**

Formato: SWAP op_1, op_2
Tipo Args: (direcc) (reg)
Lógica: **AUX** $\leftarrow op_2$
$op_2 \leftarrow MEMORIA\ (op_1)$
$op_1 \leftarrow$ **AUX**
Descripción: Intercambia los valores entre la palabra (4 bytes) almacenada en la dirección op_1 en el registro op_2 .
Banderas: No modifica las flags.

- **SETHI**

Formato: SETHI op_1, op_2
Tipo Args: (const22) (reg)
Descripción: Coloca en los 22 bits más significativos de op_2 el valor op_1.
Banderas: No modifica las flags.

BIFURCACIÓN INCONDICIONAL

- **CALL**

Formato: CALL op_1
Tipo Args: (desp30)
Lógica: %15 = PC
PC = PC + 4*desp30
Descripción: Coloca en el registro %15 el valor actual del PC (es decir de la instrucción que contiene el CALL) y realiza un salto incondicional a la dirección PC+4*desp30
Banderas: No modifica las flags.

- **JMPL**

Formato: JMPL op_1, op_2
Tipo Args: (address) (reg)
Lógica: op_2 = PC
PC = op_1
Descripción: Coloca en el registro op_2 el valor actual del PC (es decir de la instrucción que contiene el JMPL) y realiza un salto incondicional a la dirección op_1.
Banderas: No modifica las flags.

- **RETT**

Formato: RETT op_1
Tipo Args: (direcc)

Lógica: Si ET = 1 ⇒ { interrupciones habilitadas }
TRAP
sino si (ET=0 y no en modo supervisor) o
(window overflow) o
(error de alineación) ⇒
TRAP
sino
$ET \leftarrow 1$
PC = op_1
fin si

Descripción: Es usada para retornar de un servicio a interrupción. Verifica que el retorno sea válido y en caso de serlo transfiere el control a la dirección op_1.

Banderas: No modifica las flags.

INSTRUCCIONES PARA MANEJO DE VENTANAS

SAVE

Formato: SAVE op_1, op_2, op_3

Tipo Args: (reg) (*reg_o_imm*) (reg)

Lógica: $new_CWP = (CWP - 1) \bmod N_WINDOWS$
Si WIM[new_CWP] ⇒
genera trap window_overflow
Sino
$op_{3_{new_win}} = op_{1_{actual_win}} + op_{2_{actua_win}}$
CWP = new_CWP
Fin si

Descripción: Guarda la ventana del procedimiento llamador.
Coloca en el registro op_3 ***de la nueva ventana*** el valor $op_1 + op_2$ ***calculado a partir de los valores de los registros de la ventana vieja***.

Banderas: No afecta las flags.

RESTORE

Formato: RESTORE op_1, op_2, op_3

Tipo Args: (reg) (*reg_o_imm*) (reg)

Lógica: $new_CWP = (CWP - 1) \bmod N_WINDOWS$
Si WIM[new_CWP] ⇒
genera trap window_overflow
Sino
$op_{3_{new_win}} = op_{1_{actual_win}} + op_{2_{actua_win}}$
CWP = new_CWP
Fin si

Descripción: Guarda la ventana del procedimiento llamador.
Coloca en el registro op_3 ***de la nueva ventana*** el valor $op_1 + op_2$ ***calculado a partir de los valores de los registros de la ventana vieja***.

Banderas: No afecta las flags.

BIFURCACIÓN CONDICIONAL

BA (Branch Always)

Formato: BA op_1
Tipo Args: (desp22)
Lógica: Si 1 $\Rightarrow$
PC = PC + 4 *extender_ signo(op_1)
Descripción: Esta instrucción es en realidad un salto incondicional pero es equivalente en formato a todas las demás que realizan saltos condicionales en el sentido de realizar un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

BNE (Branch Not Equal)

Formato: BNE op_1
Tipo Args: (desp22)
Lógica: Si *not* Z $\Rightarrow$
PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag de Zero sea 0 realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

BE (Branch Equal)

Formato: BE op_1
Tipo Args: (desp22)
Lógica: Si Z $\Rightarrow$ PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag de Zero sea 0 realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

BG (Branch Greater)

Formato: BG op_1
Tipo Args: (desp22)
Lógica: Si *not* $[Z \vee (N \otimes V)] \Rightarrow$ PC = PC + 4 *extender_ signo(op_1)
Descripción: *not* $[Z \vee (N \otimes V)] \equiv$ *not* $Z \wedge$ *not* $(N \otimes V) \equiv$ not $Z \wedge (N = V)$.
Si la condición anterior se cumple entonces implica que en caso de haber sido realizada una resta (una comparación) A-B de dos números A y B codificando enteros en complemento a 2 entonces $A \neq B$ y $A \geq B$ entonces $A > B$ y se realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

BLE (Branch Less Or Equal)

Formato: BLE op_1
Tipo Args: (desp22)
Lógica: Si $Z \vee (N \oplus V) \Rightarrow$ PC = PC + 4 *extender_ signo(op_1)
Descripción: La condición es la negación del caso anterior (ver BG) es decir que el salto se realiza si $A \leq B$ considerando el signo.
Banderas: No modifica las flags.

BGE (Branch Greater Or Equal)

Formato: BGE op_1
Tipo Args: (desp22)
Lógica: Si *not* (N ⊕ V) ⇒ PC = PC + 4 *extender_ signo(op_1)
Descripción: La condición se cumple si N = V y por lo tanto es verdadera cuando A ≥ B considerando el signo (ver BG) y en ese caso se realiza el salto.
Banderas: No modifica las flags.

BL (Branch Less)

Formato: BL op_1
Tipo Args: (desp22)
Lógica: Si N ⊕ V ⇒ PC = PC + 4 *extender_ signo(op_1)
Descripción: La condición es la negación del caso anterior (ver BGE) y por lo tanto el salto se produce si A < B (ver BG) considerando el signo.
Banderas: No modifica las flags.

BGU (Branch Greater Unsigned)

Formato: BGU op_1
Tipo Args: (desp22)
Lógica: Si *not* C ∧ *not* Z ⇒ PC = PC + 4 *extender_ signo(op_1)
Descripción: Si la condición anterior se cumple entonces implica que en caso de haber sido realizada una resta (una comparación) C-D de dos números C y D codificando enteros sin signo entonces C ≠ D y C ≥ D entonces C > D y se realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

BLEU (Branch Less Or Equal Unsigned)

Formato: BLEU op_1
Tipo Args: (desp22)
Lógica: Si C ∨ Z ⇒ PC = PC + 4 *extender_ signo(op_1)
Descripción: Es la negación de la condición anterior es decir que se realiza el salto si C ≤ D (ver BGU) sin considerar el signo de C y D.
Banderas: No modifica las flags.

BCC (Branch On Carry Clear)

Formato: BCC op_1
Tipo Args: (desp22)
Lógica: Si *not* C ⇒ PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag de Carry sea 0 realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1. Es equivalente a C ≥ D (ver BGU).
Banderas: No modifica las flags.

BCS (Branch On Carry Set)

Formato: BCS op_1
Tipo Args: (desp22)
Lógica: Si C ⇒ PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag de Carry sea 1 se realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1. Es equivalente a C < D (ver BGU).
Banderas: No modifica las flags.

- **BPOS (Branch Positive)**

Formato: BPOS op_1
Tipo Args: (desp22)
Lógica: Si *not* N ⇒ PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag Negative sea 0 realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

- **BNEG (Branch Negative)**

Formato: BNEG op_1
Tipo Args: (desp22)
Lógica: Si N ⇒
PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag Negative sea 1 realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

- **BVC (Branch On Overflow Clear)**

Formato: BVC op_1
Tipo Args: (desp22)
Lógica: Si *not* V ⇒
PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag de Overflow sea 0 realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

- **BVS (Branch On Overflow Set)**

Formato: BVS op_1
Tipo Args: (desp22)
Lógica: Si V ⇒
PC = PC + 4 *extender_ signo(op_1)
Descripción: En caso que la flag de Overflow sea 1 realiza un salto a la dirección PC+4*op_1 considerando el signo de op_1.
Banderas: No modifica las flags.

MANEJO DE REGISTROS ESPECIALES

- **PSR (Program Status Register)**

n	z	v	c
23	22	21	20

Los bits del 20 al 23 del PSR almacenan las banderas de la IU. Estos bits son modificados por las instrucciones con cc (por ejemplo ANDcc) y por la instrucción WRPSR.

- **RDPSR**

Formato: RDPSR op_1
Tipo Args: (reg)

Lógica: $op_1 \leftarrow$ PSR
Descripción: Copia el PSR a op_1.
Banderas: No modifica las flags.

- **WRPSR**

Formato: WRPSR op_1, op_2
Tipo Args: (reg) (reg_o_imm)
Lógica: PSR $\leftarrow op_1 \oplus op_2$
Descripción: Coloca en PSR el valor op_1 op_2 1 2 Å .
Banderas: Modifica todas las flags ya que estas son bits del PSR.

- **RDWIM**

Formato: RDWIM op_1
Tipo Args: (reg)
Lógica: $op_1 \leftarrow$ WIM
Descripción: Copia el WIM (Window Invalid Mask) a op_1.
Banderas: No modifica las flags.

- **WRWIM**

Formato: WRWIM op_1, op_2
Tipo Args: (reg) (reg_o_imm)
Lógica: WIM $\leftarrow op_1 \oplus op_2$
Descripción: Coloca en WIM el valor $op_1 \oplus op_2$.
Banderas: No modifica las flags.

- **RDTBR**

Formato: RDTBR op_1
Tipo Args: (reg)
Lógica: $op_1 \leftarrow$ TBR
Descripción: Copia el TBR (Trap Base Register) a op_1.
Banderas: No modifica las flags.

- **WRTBR**

Formato: WRTBR op_1, op_2
Tipo Args: (reg) (reg_o_imm)
Lógica: TBR $\leftarrow op_1 \oplus op_2$
Descripción: Coloca en TBR el valor $op_1 \oplus op_2$.
Banderas: No modifica las flags.

CARTILLA SPARC (Reducida)	
INSTRUCCIÓN	**OPERACIÓN**
ADD $reg_1,\ reg_o_imm,\ reg_2$	$reg_2 \leftarrow reg_1 + reg_o_imm$
ADDX $reg_1,\ reg_o_imm,\ reg_2$	$reg_2 \leftarrow reg_1 + reg_o_imm + c$
SUB $reg_1,\ reg_o_imm,\ reg_2$	$reg_2 \leftarrow reg_1 - reg_o_imm$
SUBX $reg_1,\ reg_o_imm,\ reg_2$	$reg_2 \leftarrow reg_1 - reg_o_imm - c$

AND $reg_1,\ reg_o_imm,\ reg_2$	$reg_2 \leftarrow reg_1 \wedge reg_o_imm$
OR $reg_1,\ reg_o_imm,\ reg_2$	$reg_2 \leftarrow reg_1 \vee reg_o_imm$
XOR $reg_1,\ reg_o_imm,\ reg_2$	$reg_2 \leftarrow reg_1 \oplus reg_o_imm$

SLL $reg_1,\ reg_o_imm,\ reg_2$	corre $reg_1,\ reg_o_imm$ bytes a la izq. y coloca el resultados en reg_2 . Rellena con ceros.
SRL $reg_1,\ reg_o_imm,\ reg_2$	corre $reg_1,\ reg_o_imm$ bytes a la der. y coloca el resultados en reg_2 . Rellena con ceros.
SRA $reg_1,\ reg_o_imm,\ reg_2$	corre $reg_1,\ reg_o_imm$ bytes a la der. y coloca el resultados en reg_2 . Extiende signo.

LDSB $direcc,\ reg$	carga en reg el byte que esta en la dirección $direcc$. Expande el signo
LDUB $direcc,\ reg$	carga en reg el byte que esta en la dirección $direcc$. No expande el signo
LDSH $direcc,\ reg$	carga en reg la media palabra que esta en la dirección $direcc$. Expande el signo
LDUH $direcc,\ reg$	carga en reg la media palabra que esta en la dirección $direcc$. No expande el signo
LD $direcc,\ reg$	carga en reg la palabra que esta en la dirección $direcc$.

STB $reg,\ direcc$	coloca los 8 bits menos significativos de reg en $direcc$
STH $reg,\ direcc$	coloca los 16 bits menos significativos de reg en $direcc$
ST $reg,\ direcc$	coloca a reg en $direcc$

SWAP $direcc,\ reg$	intercambia los valores de $direcc$ y reg

SETHI $const22,\ reg$	pone a cero los 10 bits menos significativos de reg y coloca a $const22$, en los 20 bits mas significativos .

CALL $desp30$	pasa el control a PC+4*$desp30$ y almacena el PC en %15
JMPL $direcc,\ reg$	pasa el control a $direcc$ y almacena el PC en reg

CARTILLA SPARC (Reducida)	
INSTRUCCIÓN	**OPERACIÓN**
BNE *desp22*	salta a PC+4*desp22 si no Z
BE *desp22*	" si Z
BG *desp22*	" si no Z y N=V
BLE *desp22*	" si Z o N≠V
BGE *desp22*	" si N=V
BL *desp22*	" si N≠V
BGU *desp22*	" si no C y no Z
BLEU *desp22*	" si C or Z
BCC *desp22*	" si no C
BCS *desp22*	" si C
BPOS *desp22*	" si no N
BNEG *desp22*	" si N
BVC *desp22*	" si V
BVC *desp22*	" si no V

RDPSR *reg*	coloca el valor del PSR en reg
WRPSR *reg, reg_o_imm*	coloca en PSR el resultado de *reg* xor *reg_o_imm*
RDWIM *reg*	coloca el valor de WIM en reg
WRWIM *reg, reg_o_imm*	coloca en WIM el resultado de *reg* xor *reg_o_imm*
RDTBR *reg*	coloca el valor del TBR en reg
WRTBR *reg, reg_o_imm*	coloca en TBR el resultado de *reg* xor *reg_o_imm*

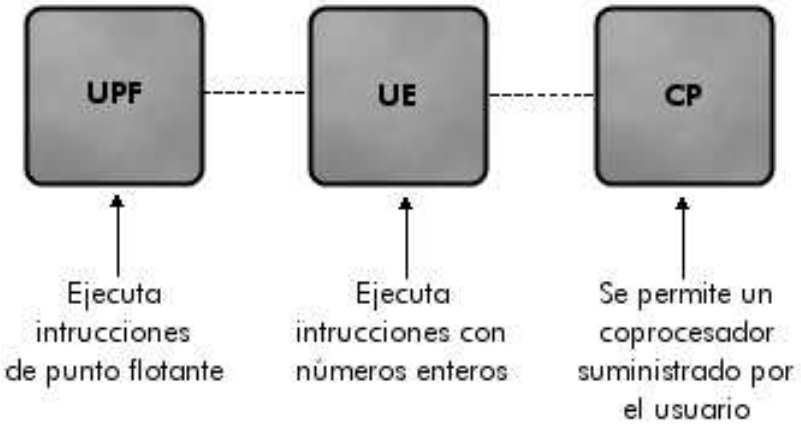
SAVE reg_1, reg_o_imm, reg_2	salva la ventana del procedimiento llamador
RESTORE reg_1, reg_o_imm, reg_2	restablece la ventana del procedimiento llamador

Descripción de sus Partes

Componentes

Un procesador SPARC comprende una Unidad de Enteros (UE), una Unidad de Punto Flotante (UPF) y un Co-Procesador opcional, cada uno de ellos con sus propios registros. Ésta organización permite una máxima coordinación entre la ejecución de instrucción de enteros, de punto flotante y de co-procesador. Todos los registros, con la posible excepción de los del co-procesador, tienen una longitud de 32 bits.

El procesador puede estar en uno de dos modos: usuario o supervisor. En el modo supervisor el procesador puede ejecutar cualquier instrucción, incluyendo aquellas privilegiadas (sólo-supervisor). En el modo de usuario, un intento de ejecutar una instrucción privilegiada causaría una trap (señal) al software supervisor. Programas de tipo “Aplicación de usuario” son aquellos que se ejecutan mientras el procesador está en modo de usuario.



Unidad de Enteros (UE)

La UE contiene los registros de propósito general y controla todas las operaciones del procesador. La UE ejecuta instrucciones aritméticas de enteros y computa direcciones de memoria para cargas y almacenamientos. También mantiene el contador de programa y controla la ejecución de instrucciones de la UPF y el CP. Una UE puede contener desde 40 hasta 520 registros *r* de propósito general de 32 bits cada uno. Esto corresponde a una agrupación de los registros en 8 registros *r* globales, más un stack circular de entre 2 y 32 sets de 26 registros cada uno, conocido como ventanas de registros.

Unidad de Punto Flotante (UPF)

La UPF tiene 32 registros de punto flotante de 32 bits cada uno. Para almacenar valores de doble precisión se utilizan 2 registros, y valores de cuádruple precisión ocupan un grupo de 4 registros adyacentes. En consecuencia, los registros de punto flotante pueden contener un máximo de 32 valores de simple precisión, 16 de doble precisión, u 8 de cuádruple precisión. Las instrucciones de carga y almacenamiento en punto flotante son usadas para mover datos entre la UPF y la memoria. La dirección de memoria es calculada por la UE.

Co-Procesador (CP)

El Co-Procesador tiene su propio set de registros de normalmente 32 bits. Instrucciones de carga/almacenamiento del Co-Procesador son las que se usan para mover datos entre los registros del Co-Procesador y la memoria. Para cada instrucción de carga/almacenamiento de punto flotante, hay una instrucción de carga/almacenamiento del Co-Procesador análoga.

DESCRIPCIÓN DEL SOFTWARE

EL AMBIENTE OPERATIVO SOLARIS

Solaris 2.5.1, solución de computación distribuida de SunSoft's, es un entorno operativo empresarial o para Internet, que se caracteriza por una integración total con servicios de Internet y Networking; viene con herramientas de desarrollo para construir aplicaciones de comunicación e información avanzadas; tiene una librería de más de 10,000 aplicaciones para todos los líderes del mercado de software.

A Solaris se le menciona como Entorno Operativo y no como Sistema Operativo por que se trata de un gran conjunto de herramientas (entre las que se encuentra el sistema operativo en sí), las cuales fueron desarrolladas con la finalidad de aprovechar al máximo el rendimiento del hardware.

Este entorno operativo le brinda todas las ventajas de multitarea, manejo de redes, multiprocesamiento simétrico, librerías multihilo, compatibilidad ABI y la ya conocida capacidad multithreading (la capacidad de realizar operaciones simultáneas dentro de una misma aplicación).

VENTAJAS QUE PROVEE EL AMBIENTE OPERATIVO SOLARIS

Portabilidad

El SunOS de SunSoft es portátil a través de múltiples plataformas de vendedores. Mediante la portabilidad ABI (Interface de Aplicación Binaria), se provee una interfase que le permite desarrollar aplicaciones UNIX compatibles en cualquier sistema Solaris. El software ABI corre como software shrink−wrapped sobre todos los sistemas de vendedores con la misma arquitectura de microprocesador. Esto permite a los desarrolladores de aplicación reducir los costos de desarrollo de software y traer productos para comercializar rápidamente, y permite a los usuarios mejorar el hardware mientras retienen sus aplicaciones de software y minimizando costos de conversión.

Escalabilidad

A través del tiempo, las aplicaciones llegan a ser más ampliamente usadas, y requieren que sistemas más poderosos los apoyen. Para operar en un Ambiente creciente, el software debe ser capaz de correr en una amplia rango de poder y debe ser capaz de tomar ventaja del poder adicional de procesamiento. El sistema Operativo de SunSoft corre sobre máquinas de todos los tamaños, desde laptops a supercomputadoras.

Interoperabilidad

Los Ambientes de computación heterogéneos son una realidad hoy. Los usuarios compran sistemas de muchos vendedores para implementar las soluciones que ellos necesitan. La estandarización y las interfases claras son críticas para un ambiente heterogéneo, permitiendo a los usuarios desarrollar estrategias para comunicar a lo largo de su red. Los sistemas Solaris pueden interoperar con cada sistema popular sobre el mercado hoy, y las aplicaciones que corren sobre UNIX pueden comunicarse fácilmente.

Compatibilidad

La tecnología de Computación continúa avanzando rápidamente, pero la necesidad de permanecer competitivo requiere que los vendedores minimicen sus costos y maximicen sus inversiones. SunSoft asegurará que tanto como la nueva tecnología es introducida, la inversión existente de software se conserve. Los usuarios pueden aprovechar soluciones de hoy y todavía ser compatibles con tecnologías de mañana.

DATOS TÉCNICOS-COMERCIALES

Características Relevantes

UltraSPARC IV

- 1 GHz, 1.2 GHz
- Diseño Chip MultiThreading
  - Dual thread
  - Subsistema de memoria mejorado
  - Mayores características RAS
- Actualización
  - Pin-compatible con UltraSPARC III
- Tecnología 130 nm, próxima generación UIV+90 nm
- Hasta 2x mayor throughput que UltraSPARC-III

UltraSPARC III

- 1 GHz, 1.2 GHz
- Proceso de fabricación
  - 29 millones de transistores
  - L1 64 KB 4-way data cache, asociativa por conjuntos
  - L1 32 KB 4-way instruction cache, asociativa por conjuntos
  - Transistores de puerta de 100 nm
  - 0.13u CMOS 6 Layer metal, cobre
- Controlador de memoria dentro del chip
  - 2.4 GB/sec ancho de banda a memoria por proc.
- Etiquetas de caché externa L2 dentro del proc
  - Chequeo de coherencia ejecutando a la veloc.
  - 9.6 GB/sec ancho de banda de coherencia

Comparación Técnica-Económica con otros Sistemas Similares

Como comparación, mencionaremos las características principales del procesador Pentium 4, y luego destacaremos de manera ampliada las especificaciones técnicas del último procesador de Sun con tecnología SPARC, el UltraSPARC IV.

Intel® Pentium® 4 Processor

- ✓ Su arquitectura esta comprendida en una tecnología que realiza procesos en 90 nm, 130 nm y 180 nm.
- ✓ Su Cache está comprendido entre los 256k, los 512k y los 1024k (1MB)
- ✓ Tiene una velocidad clock de 1.30 a 2.80 GHz
- ✓ Tiene una velocidad de bus de chipset de 400, 533 MHz
- ✓ Se utiliza para aplicaciones de fotografía digital, música y video digital, juegos, aplicaciones hogareñas de altos requerimientos, para aplicaciones multimedia, y para herramienta de productividad para negocios y empresas.

Sun® UltraSPARC IV Processor

Tipo de arquitectura:

- ✓ Chip multithreading (CMT) procesador con dos threads por procesador
- ✓ Basado en dos UltraSPARC III pipelines
- ✓ Arquitectura de 64-Bit SPARC con Set de Instrucciones VIS
- ✓ 66 millones de transistores
- ✓ 4-vias superescalar
- ✓ 14-stage nonstalling pipeline

Tecnología de procesos:

- ✓ CMOS .13 micrones de procesos
- ✓ 1368-pin flip-chip cerámico Land Grid Array (LGA)

Interconexion:

- ✓ Sun Fireplane interconectado corriendo a 150 MHz.

Frecuencia:

- ✓ Frecuencia Clock: 1.05 - 1.2 GHz

Cache:

- ✓ L1 Cache (por pipeline): 64 KB 4-vías datos
  32 KB 4-vías instrucciones
  2 KB Write, 2 KB Prefetch
- ✓ L2 Cache: 16 MB externos (exclusivo acceso a 8 MB por pipeline)
  Controlador On-chip y tags de direcciones

Escalabilidad:

- ✓ Escalabilidad de multiprocesamiento con soporte de arquitectura para más de 1000 procesadores/sistema.

Memoria:

- ✓ Memoria Máxima: Subsistema de 16GB de memoria por procesador.
- ✓ Controlador de Memoria: Controlador de memoria On-chip capaz de direccionar hasta 16 GB de memoria principal a 2.4 GB/s.

I/O:

- ✓ Interfaz de Bus: Interconexión Sun Fireplane.

Alimentación:

- ✓ Consumo de energía: Un máximo de 108 watts a 1.2 GHz, 1.35 volts.

Sistemas:

- ✓ Desarrollado en:
  - o Servidores Sun Fire High-End
    - ▪ Servidor Sun Fire E25K
    - ▪ Servidor Sun Fire E20K
  - o Servidores Sun Fire Midframe
    - ▪ Servidor Sun Fire E2900
    - ▪ Servidor Sun Fire E4900
    - ▪ Servidor Sun Fire E6900

Precios

Establecer un precio para un único procesador UltraSPARC IV es bastante difícil debido a que estos se encuentran insertos generalmente en Mainboards, incluyendo a veces memorias. Se puede estimar que el precio por unidad de estos procesadores oscila alrededor de los U$S 10000.

En el mercado se ofrecen equipos que pueden o no traer incorporadas memorias, e incluso pueden poseer plataformas que incluyen más de un microprocesador. Los precios de estos elementos son los siguientes:

- ✓ COD Uniboard con 4 UltraSPARC IV Procesadores de 1.2-GHz sin licencia, con 16 MB de cache cada uno & 32 GB de memoria: U$S 141,000
- ✓ Placa de CPU/Memoria con 2 UltraSPARC IV Procesadores de 1050-MHz, sin memoria: U$S 22,995
- ✓ Placa de CPU/Memoria con 2 UltraSPARC IV Procesadores de 1.2-GHz, sin memoria: U$S 25,800
- ✓ Uniboard con 4 Procesadores UltraSPARC IV de 1.2-GHz, con 16 MB de cache cada uno y 16 GB de memoria: U$S 170,000
- ✓ Uniboard con 4 Procesadores UltraSPARC IV de 1.2-GHz, con 16 MB de cache cada uno y 32 GB de memoria: U$S 201,000

Comparar el precio de un CPU UltraSPARC con un CPU Pentium IV, el cual se encuentra en el mercado a U$S 190, sería bastante absurdo ya que la diferencia de valores es enorme. También debe considerarse que los usos que se le da a cada uno de ellos son completamente diferentes, igual que sus prestaciones.

Recomendaciones para la Sección

La densidad y versatilidad de los procesadores UltraSPARC hacen que sean una muy buena elección para muchas aplicaciones, abarcando desde estaciones de escritorio y pequeños servers de empresas hasta servers "blade". El enfoque principal se hace sobre aplicaciones de empresa que requieren un performance escalable y puede aprovechar ventajosamente un multiprocesamiento, un cache on-chip más grande, memoria más amplia y un ancho de banda de Input/Output (Entrada/Salida) incrementado.

CONCLUSIONES

La configuración del procesador UltraSPARC lo hace adecuado para la carga de programas orientados tanto a grandes cantidades de datos como a grandes cantidades de transacciones. El sistema de 4-vías es ideal para aplicaciones como:

- ✓ Procesos de transacción on-line
- ✓ Codificación y decodificación de archivos Multimedia
- ✓ Traslado de datos
- ✓ E-Comercio
- ✓ Manejo de mensajes (e-mail)
- ✓ Aplicaciones de gráficos e imágenes
- ✓ Manejo de redes
- ✓ Planificación de Recursos de Empresas

BIBLIOGRAFÍA

www.sun.com
www.monografías.com
http://web.frm.utn.edu.ar/arquitectura2
TANENBAUM, Andrew. *Organización de Computadoras: Un Enfoque Estructurado.* Editorial Prentice Hall, 1992, Tercera Edición.